Chapter 6

Design Projects

6.1 Suggested Project 1: Power system optimization

Brief highlights

- Solve a relatively realistic power systems optimisation problem.
- Compete with your classmates for the best solution (bonus marks awarded).
- MATLAB or Python software used; evaluation and (sub-optimal) solution code provided as a starting point.
- High flexibility: a wide range of possible approaches are possible.

6.1.1 Introduction

In this assignment, your task will be to design a function to optimize the build and operation of generation (and storage if so desired) for a given power system. The aim is to maximize cash (i.e. profits plus any starting cash left unused; excluding asset value) at the end of a year-long simulation.

In the build stage, your programme will be given details of the system and parameters for each type of generation and decide what to build. In the operation stage, your second module will decide how much generation (of each type) to supply to the system for each hourly time period.

MATLAB and Python code to run the year-long simulation is provided on Learn, along with an example (sub-optimal) solution. Either language is fully acceptable and will result in equivalent performance in the final evaluation. Those exceedingly short of time have the option only to optimize the build process while using the given operation code. Such solutions should not expect the very highest marks, but a good result is still possible.

The following sections explain the problem in detail. However, please note that the code and associated comments may explain the problem more clearly than any written text. It is also **not** necessary to understand the simulation detail below - one could modify the provided code into an objective function and optimize the any/all parameters by any appropriate method without using intuition or other knowledge.

Generation	Build cost ($ millions / MW)	Capacity factor
Wind	$\mathcal{N}(1.1,\ 0.11^2)$	Uniform $\in$ [0 50%]
Geothermal	$\mathcal{N}(4.8\ 0.48^2)$	100%
Hydro	$\mathcal{N}(2.8,\ 0.28^2)$	$\mathcal{N}(45\%,\ 4.5\%^2)$
Solar	$\mathcal{N}(0.8,\ 0.08^2)$	Average 24.58%

Table 6.1: Generation options

6.1.2 Notation

We will denote a Gaussian random variable X with mean μ and standard deviation σ by $X \sim \mathcal{N}(\mu, \sigma^2)$.

6.1.3 Problem formulation

Generation options

Four generation options are provided:

1. **Wind** is modelled by a random wind capacity factor from 0.00 to 0.50, i.e. the available wind power ranges from 0 to 50% of built capacity (uniform distribution). This value changes at each time period.

2. **Hydro** is fully controllable up to its rated power. However, the overall capacity factor throughout the year is limited to a constant value which is set for each year-long simulation via a Gaussian random variable with 0.45 mean and 0.045 standard deviation. When the cumulative total of hydro exports reaches this number, it is assumed that the hydro generator is out of water and no further generation is permitted. For simplicity, in-flows and out-flows etc. are not explicitly modelled.

3. **Geothermal** is fully controllable up to its rated power.

4. **Solar** is modelled by a 24-hour cycle capacity factor from 0.00 to 0.80 with 20% multiplicative randomness at each time period (uniform distribution). Full details are given in the provided code.

Table 6.1 shows the parameters for each generation option. Each Gaussian random variable has 10% standard deviation. Each uniform random variable is given bounds.

Storage

Battery storage is available with the following parameters:

- The cost is $150,000 per MWh. This is randomized for each year-long simulation via a Gaussian random variable with 10% standard deviation.
- It is assumed that the rated power in MW is equal to the rated capacity in MWh, i.e. a charge / discharge time of one hour.
- One-way efficiency is 97.5%.
- The usable state-of-charge is from 0% to 100%.
- Batteries are fully charged at the start of the simulation.

Load

The system mean load L_{avg} is chosen randomly for each year-long simulation, uniformly distributed between [1000,2000] MW. For each time-period, the load at that time period is a Gaussian random variable with the mean as chosen above and a standard deviation of 500 MW, i.e. `load(t)` $\sim \mathcal{N}(L_{avg},\ 500^2)$ MW.

Pricing

The market price will be set by the following formula at each time period t:

```
base_price(t) = base_price_average + base_price_average/10*randn
price(t) = base_price(t) + load(t)*priceCoefficient1
                         - wind_capacity_factor(t)*priceCoefficient2
```

where `base_price_average` $\sim \mathcal{N}(150,\ 15^2)$, `priceCoefficient1` $\sim \mathcal{N}(0.075,\ 0.0075^2)$, and `priceCoefficient2` $\sim \mathcal{N}(150,\ 15^2)$ are chosen randomly for each year-long simulation. This price (in $ per MWh) will be multiplied by the amount (in MWh) you decide to generate, and the result added to your funds.

If your programme does not return sufficient generation, the interface will penalize this as follows:

- A shortfall of up to `external_MWs(t)` will be purchased at market price.
- Any further shortfall cannot be supplied and a penalty cost of $1000 per MWh will be deducted from your funds.

If your programme returns excess generation, this is curtailed / spilled for free.

6.1.4 Solution methodology

Possible approaches include gradient-based methods, particle swarm optimization, neural networks, etc. Computational time may be a limitation, and it will be imperative to explain your approach and design choices clearly.

A solution has two parts:

1. A module `build_plants()` to decide how much of each type of power plant to build at the start of each year-long simulation. The interface will check if the proposed build is feasible, and throw an error if not.
2. A module `operate_plants()` to decide how much generation of each type to offer at each time-step. For solar, wind and geothermal generation, this is trivial as there is no benefit to reducing output (since curtailment is free). However, it is non-trivial to work out when battery storage should charge or discharge, and (given the finite amount of hydro energy) when to use hydro to avoid shortfalls.

Only a few constraints are placed upon your solution:

- No forecast or future information will be given, i.e. at a time period T, information from a future time-step $t > T$ will not be known.

- `operate_plants()` should execute in less than 1ms on a standard computer in the computer lab. If your code is close to this (this should not be an issue in most cases), please time your code and include the info in your report. You may use any method, e.g. `(https://au.mathworks.com/help/matlab/ref/timeit.html)` or the Profiler tool in MATLAB, or *profile* or *cProfile* in Python.

- `build_plants()` should execute in less than 1s on a standard computer in the computer lab. If your code is close to this (this should not be an issue in most cases), please time your code and include the info in your report. You may use any method as per the above.

In order to get you started, a demonstration solution has been provided:

- `build_plants()` spends 20% of its money building each of wind, geothermal, hydro, solar, and battery storage, regardless of given costs and other parameters. This is far from optimal.

- `operate_plants()` uses simple logic to avoid a shortfall whenever possible. Wind, solar and geothermal generations are run at their maximum available capacity at that point in time. If there is still a shortfall, any hydro generation is used to cover the shortfall, and if there is still a shortfall, any stored energy in the battery is used to reduce or (if possible) avoid the shortfall. There is also considerable room for improvement in this module.

Hints

1. Not all the starting funds need to be used building generation and/or storage. Since the objective is to maximize cash at the end of one year, it may be beneficial to under-invest. Any unused starting funds will be added to your cash flow at the end to make your final score.

2. Because there are penalties for failing to supply load (as in the real world), it may be beneficial to have a mix of generation and/or storage ...

3. The ideal mix of generation and/or storage is likely to depend on the loading, costs, and capacity factors which are randomized each year-long simulation. You may wish to adjust your build plans based on this info which is passed to `build_plants()`, as opposed to sticking to fixed proportions per the demo solution.

4. As is often the case for power system optimization problems, the global optimization problem may be computationally impractical to solve. Firstly, there are a number of stochastic (random) variables, the exact value of which is not known in advance. Secondly, there are 8760 time periods, and for each time period, generation + storage MW contributions are design variables (i.e. they need to be optimized), potentially resulting in over 40,000 design variables and even more constraints. Hence, it may be helpful to look for ways to simplify or decompose the problem.

6.1.5 Deliverables

The progress report should cover the following: a little background / intuition about the problem; a short overview of possible methods to use for the problem; a plan for

Item	Weight
Presentation	0.25
Quality of the technical design	0.50
Performance	0.25

Table 6.2: Final report marking

your solution; any progress-to-date. It is very easy to find a solution, even by trial and error, better than the given starting solution, so it is expected to show some preliminary improvement by the time of the progress report.

The final report should explain your solution in-depth (see below), and be accompanied by the MATLAB or Python code used to implement your solution. Please email this to me at jeremy.watson@canterbury.ac.nz or upload to the Learn dropbox. Your solution will be tested in the given environment and (per the next section) some marks will be awarded for performance.

6.1.6 Marking

The final report will be assessed as in Table 6.2:

Guidelines

Please take note of the following:

Presentation: Reports should be tidy and concise. Use graphs / figures as appropriate to illustrate your solution. Any format is acceptable, although IEEE conference format (https://www.ieee.org/conferences/publishing/templates.html) is suggested. It is important to focus on the parts you developed, as opposed to explaining the problem and/or provided code.

Quality of the technical design: Explain what methods you tried and justify your final choice. Solutions which attempt to optimize both the build and operation process are encouraged, and should demonstrate benefit compared to only optimizing the build using the provided operation code. Comparisons of different optimisation approaches are encouraged.

Performance: you should test your code with the provided evaluation function (on LEARN) and report its score in your final report. I will independently evaluate your solution by changing the rng seed and running the simulation with `totalSimulations` equal to 100, from which the average score will be taken. Marks will be awarded as follows:

1. The best solution from the class will score full marks for this section, so long as it achieves a score of $6.5 billion or better.

2. Marks thereafter will be awarded based on the average score, using linear interpolation where the best score is 100 % and the class median score is set at 70%. However, regardless of this calculation, no negative marks will be awarded; all complete and working solutions better than the starting solution will score at least 30% for this section.

3. Bonus marks of 10% (of the marks assigned to performance) will be awarded for any solution which outperforms my own sub-optimal solution. If this results in a mark exceeding 100%, the mark will not be capped.